Relatório Técnico de Laboratório Prático de Infraestrutura de Redes como Serviço

Aluno: João Victor Póvoa França

Palmas - TO

Resumo

Este relatório técnico descreve a arquitetura, o desenvolvimento e a implementação da plataforma de monitoramento e telemetria my-telemetry. O projeto visa simular os elementos críticos de ferramentas clássicas de gerenciamento de redes, como Zabbix e SNMP, provendo uma biblioteca customizada de agenciamento de métricas, um servidor alvo instrumentado e um *dashboard* analítico em tempo real. Utilizando uma arquitetura de *monorepo* (*NPM Workspaces*) e a biblioteca systeminformation, o sistema coleta dados vitais de *hardware* — como carga de CPU e consumo de memória RAM — e os transporta via protocolo HTTP. O documento também explora a dualidade e os *trade-offs* entre os protocolos HTTP (REST) e SNMP, além de apresentar os resultados de testes de estresse que validam a eficácia dos alertas de infraestrutura em cenários de alta carga computacional.

Introdução

No contexto atual de Infraestrutura de Redes como Serviço (IaaS, PaaS e SaaS), o gerenciamento de redes é um conjunto de processos essencial para monitorar, controlar, manter e otimizar os recursos computacionais. A observabilidade contínua é fundamental para garantir o cumprimento de SLAs (*Service Level Agreements*), bem como a escalabilidade e a alta disponibilidade das aplicações. De acordo com as diretrizes abordadas na disciplina, essas práticas alinham-se diretamente ao modelo estrutural FCAPS, com destaque especial para as áreas de Gerenciamento de Desempenho (*Performance Management*) e Gerenciamento de Falhas (*Fault Management*).

Neste cenário, este relatório documenta o desenvolvimento e a implementação da biblioteca de monitoramento `my-telemetry`, desenvolvida como parte do laboratório prático da disciplina. A plataforma atua como uma solução customizada para a extração de métricas de infraestrutura, operando como uma alternativa moderna em HTTP aos tradicionais agentes do protocolo SNMP. O projeto busca fornecer uma interface simplificada para a coleta em tempo real do uso de CPU e memória , permitindo a rápida detecção de anomalias com o objetivo final de isolar problemas e minimizar o Tempo Médio de Reparo (MTTR) nas infraestruturas de rede.

O objetivo central é encapsular a complexidade da coleta de dados do Sistema Operacional (SO) através da criação de um agente independente (@my-infra/agent), que atua em paralelo a um servidor Node.js simulado, transmitindo seus dados a uma interface React capaz de acusar anomalias e degradação de recursos em tempo real.

O código-fonte integral foi disponibilizado de forma aberta num repositório do GitHub (https://github.com/JoaoIto/telemetry-monitoring), permitindo o controlo de versão e a consulta da arquitetura. Além disso, a biblioteca resultante foi empacotada e publicada publicamente no registo NPM sob o nome my-telemetry (https://www.npmjs.com/package/my-telemetry), permitindo a instalação remota em qualquer ambiente Node.js."

Desenvolvimento

A arquitetura do sistema destaca-se pelo isolamento de processos e pela leveza da biblioteca de telemetria. O pacote `@my-infra/agent` sobe um servidor HTTP nativo em uma porta paralela dedicada (9090), escutando o *endpoint* `/metrics`. Esse distanciamento da aplicação principal previne que gargalos nas rotas de negócio interfiram na observabilidade da rede.

O motor de coleta extrai simultaneamente informações vitais do SO de forma assíncrona, capturando indicadores como ocupação do processador (em %), memória RAM ativa, temperatura térmica, velocidade de rede na interface Ethernet e ocupação de disco. Em seguida, os bytes brutos são higienizados e convertidos em um payload JSON formatado, contendo inclusive *timestamps* Unix para ordenação temporal no cliente.

Na extremidade visual, o *Dashboard* opera sob o modelo de *Pulling Contínuo (Polling)*. Através de temporizadores (`setInterval`), a interface web requisita os dados do agente a cada 2000 milissegundos, gerando uma janela deslizante (*Sliding Window*) de 30 pontos temporais que alimentam os gráficos em linha imediatamente.

Durante o desenvolvimento, também foi possível estruturar um comparativo prático de transporte de rede:

- **HTTP (TCP):** Orientado a conexão, entrega integridade estruturada em JSON e facilidade de leitura (tipagem dinâmica e metadados ricos), porém com maior *overhead* devido ao *3-Way Handshake*.
- **SNMP (UDP):** Transporte ultraleve ideal para equipamentos restritos (roteadores, IoT). Trabalha com MIBs rigorosas, exigindo a conversão de *floats* dinâmicos para inteiros limitados (ex: `Integer32`), economizando significativamente a banda trafegada na infraestrutura, ao custo da ausência de retransmissão em caso de perda de pacotes.

Resultados

A implantação do ecossistema demonstrou pleno sucesso operacional. A empacotadora do *monorepo* gerou uma distribuição ágil da solução, que foi posteriormente publicada no registro NPM sob o nome my-telemetry. A execução unificada do comando de inicialização permitiu levantar os sensores SNMP, a API REST e a interface web perfeitamente sincronizados.

Para validar as ferramentas analíticas da plataforma, conduziu-se um teste de estresse utilizando a biblioteca de *benchmarking* autocannon. Foram disparadas 100 conexões pesadas e simultâneas durante 30 segundos contra a rota /heavy-task do servidor alvo. Essa injeção de carga resultou em um bloqueio instantâneo do *Event Loop* do Node.js hospedeiro, alavancando o consumo de processamento.

```
Running 30s test @ http://localhost:3000/heavy-task
100 connections

running [======              ] 30%(node:32632) TimeoutNegativeWarning: -6 is a negative number.
Timeout duration was set to 1.
(Use `node --trace-warnings ...` to show where the warning was created)
```

Stat	2.5%	50%	97.5%	99%	Avg	Stdev	Max
Latency	431 ms	4882 ms	9888 ms	9888 ms	5048.48 ms	2779.14 ms	9888 ms

Stat	1%	2.5%	50%	97.5%	Avg	Stdev	Min
Req/Sec	0	0	0	4	1,27	1,81	3
Bytes/Sec	0 B	0 B	0 B	1.01 kB	320 B	457 B	759 B

```
Req/Bytes counts sampled once per second.
# of samples: 30

400 requests in 30.26s, 9.61 kB read
262 errors (262 timeouts)
```

O *Dashboard*, através do seu modelo de reatividade, detectou imediatamente a saturação, e o gráfico superou a linha de corte estabelecida na regra de negócio do painel (> 80% de carga de CPU), ativando os alertas visuais dinâmicos ("Alerta Vermelho") projetados para acusar incidentes de infraestrutura à equipe de operações.

Monitoramento Simultâneo Híbrido

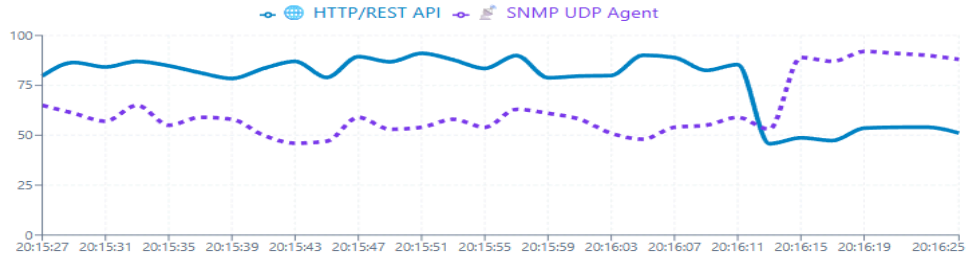
Agente em: `http://localhost:9090/` | UDP `1611`

Entender a Diferença

Telemetria Ativa

Comparativo de Coleta: Carga de CPU

51% 88%



Tráfego de Rede

HTTP	SNMP (UDP)
RX: 1.3 KB/s	RX: 0.0 KB/s
TX: 4.9 KB/s	TX: 0.0 KB/s

Alocação de Memória RAM

VIA HTTP API	VIA AGENTE SNMP
13377 MB	13387 MB

DISCO LOCAL (COMPARATIVO)

USO VIA HTTP 87%

USO VIA SNMP 87%

HOST OS (APENAS HTTP)

Sistema
Microsoft Windows 11 Home Single Language 10.0.26200 (Windows)

Processador
Intel Gen Intel® Core™ i7-13650HX (14 Cores)

Uptime da Máquina
7h 6m

Conclusão

O desenvolvimento e a implementação da plataforma `my-telemetry` atingiram com sucesso os objetivos propostos para o laboratório prático de Infraestrutura de Redes como Serviço. A ferramenta demonstrou ser plenamente eficaz na esfera do Gerenciamento de Desempenho, sendo capaz de coletar e analisar métricas vitais do servidor de forma ininterrupta. Durante os testes práticos de estresse — simulando um aumento massivo de tráfego e carga de processamento — o sistema validou sua capacidade de gerar alertas instantâneos perante o esgotamento de recursos da interface monitorada.